



SYBAGO

THE OPERATOR'S FIELD GUIDE

Build \$10k *Websites* With Claude Code.

How I shipped a portfolio of contractor websites — and a real software company — on \$44/month of tools, zero coding background, and one terminal window.

JT FASULO
FOUNDER · SYBAGO · JTFASULO.COM

FIRST EDITION
V1.0 · 2026

ABOUT THIS GUIDE

This is the playbook I use, every week, to build production websites with Claude Code. Every technique in here is taken from a real project — Sybago, jmartin-construction, coachfish, prominent-detailing, realtor-website, and a half-dozen others. No theory. No best practices I invented in a vacuum.

WHO I AM

I'm JT. I started building websites the same way you probably are now — sitting in front of a terminal, not knowing what to type. I'd never written a line of React. I'd barely written a line of HTML. Two years later I'm running a software company (Sybago), a portfolio site (jtfasulo.com), and a side business making websites for trades. Same hands. Same brain. Just a different tool.

HOW TO READ IT

Front to back the first time. After that, skip around. Part I gets you set up. Part II explains the mental model so the rest stops feeling like magic. Parts III–IX are the meat — workflow, skills, components, video, SEO, APIs. Part X walks through building a real site from a cold start.

LEGAL

© 2026 JT Fasulo / Sybago. Single-reader license. Do not redistribute. The tool prices, screenshots, and APIs cited were accurate at the time of writing; the open web moves fast, so verify before you build.

CONTENTS

What's in this book.

Forty-two chapters across ten parts. Every chapter is short — between two and six pages — so you can read on a coffee break and have something to try right after.

PART I — THE STACK

01	The four tools you actually need (\$44/month total)	14
02	Installing and signing into Claude Code	18
03	Setting up Vercel (where your sites will live)	21
04	Setting up Higgsfield AI (for cinematic hero video)	24
05	21st.dev — the free component library that does half the work	26

PART II — HOW ANY OF THIS WORKS

06	Claude Code is a junior dev who reads minds, not magic	29
07	How Vercel turns "git push" into a live URL	33
08	Where your files live: the workspace anatomy	36

PART III — THE WORKFLOW

09	Starting a new site from absolute zero	40
10	Thinking in passes — the mental shift that fixed my output	44
11	The language tricks that actually move Claude	48
12	Commit early, commit often, push every time	52

PART IV — SKILLS (THE UNFAIR ADVANTAGE)

13	What "skills" are, and why I install one per problem	56
14	frontend-design — escaping generic AI slop	59
15	animation-performance — the ten rules that killed my lag	63
16	ai-creator-video-style and branded-carousel	68

PART V — DESIGNING WITH 21ST.DEV

17	How I browse 21st.dev for inspiration, not copy-paste	72
18	Why I rewrite every imported component by hand	75
19	Three real examples — marquee, hover-button, zoom tile	78

PART VI — HIGGSFIELD AI FOR SCROLL ANIMATIONS

20	What kind of video Higgsfield is great at (and what it isn't)	82
21	My exact prompt patterns	85
22	Wiring video into a hero without killing Lighthouse	88
23	Compressing, hosting, and the Vercel limit you'll hit	91

PART VII — SEO THAT ACTUALLY WORKS

24	The title-tag pattern that ranks contractor sites	94
25	Description, Open Graph, Twitter card	97
26	JSON-LD schema — copy/paste templates	100
27	The hidden-h1 / visual-h1 split — my one weird SEO trick	100
28	Sitemap, robots, canonical, theme-color	100

PART VIII — SERVERLESS APIS ON VERCEL

29	Lead capture: /api/lead.js	110
30	Claude-powered concierge chat: /api/chat.js	110
31	Gated review widget: /api/review.js	110
32	Environment variables — keeping keys out of git	120

PART IX — SHIPPING IT

33	vercel.json — the only config file you need	120
34	Clean URLs, redirects, security headers	120
35	Connecting a custom domain	120
36	Preview deploys and the safety net you're not using	130

PART X — A FULL WALKTHROUGH

37	Day 1 — first prompt to a finished hero	13
38	Day 2 — services, testimonials, contact, footer	14
39	Day 3 — mobile pass, perf pass, deploy	14

CLOSING

40	Ten mistakes I made so you don't have to	14
41	What I learned building ten of these	15
42	Where to go from here — resources and links	15

PART ONE

The Stack.

Four tools. Forty-four dollars a month. The whole machine that produced every site in my portfolio. We'll get you signed into each one before we touch a line of code.

The four tools you actually need.

There are no agencies behind any of my sites. No designers. No developers. Four logins, totaling \$44 a month, do all of it.

When people ask me what I use to build websites I always disappoint them. They expect a list of forty Chrome extensions, a Notion stack, a designer's repertoire of plugins. What I actually use fits on a Post-it.

\$20 / MONTH · PAID

Claude Pro (with Claude Code)

The actual hands building your site. A terminal-based AI that reads, writes, and edits files for you. The Pro plan unlocks Claude Code's higher rate limits — without it you'll hit the wall mid-project.

claude.com/code

\$24 / MONTH · PAID

Higgsfield AI

For the cinematic hero videos that make a website feel like a film. Generates 5–10 second loops from a prompt or reference image. The cheap secret behind every "how is your site this beautiful?" reaction I get.

higgsfield.ai

\$0 · FREE FOREVER

Vercel (Hobby Plan)

Where your sites will actually live. Free for personal projects. You push code to GitHub; Vercel turns it into a live HTTPS URL in seconds. You'll never touch a server.

vercel.com

\$0 · FREE FOREVER

21st.dev

An open library of React components — sliders, marquees, scroll animations, hero treatments — that other people have already built and made public. The free reference catalogue I steal from constantly.

21st.dev

HONEST MATH

\$44 a month is the floor. You'll probably add a domain (\$12–\$30/year), maybe a Resend account for transactional email (\$0–\$20/mo), maybe a Stripe account (free unless you sell something). But everything in this book gets built on the \$44.

Why these four?

Each one does a job none of the others can. Claude Code is the worker. Vercel is the address. Higgsfield is the photographer. 21st.dev is the lookbook. Take any of them out and the loop breaks.

I tried for months to compress the stack. I tried Cursor instead of Claude Code (different mental model, worse for end-to-end shipping). I tried Netlify instead of Vercel (good — but Vercel's tighter Git → URL loop won me back). I tried Runway and Pika instead of Higgsfield (less direction, slower iteration). I tried sourcing components from Aceternity, ShadCN, and a half-dozen other galleries — and I still use those too — but 21st.dev kept showing up because the components are practical, not decorative.

What I'm not including in the \$44

- **A code editor.** You don't need one. Claude Code edits files for you. I have VS Code open sometimes, but only to look — never to type.
- **A design tool.** Figma is great. I don't open it. Claude Code, plus 21st.dev for components, plus screenshots of sites I like, covers the design loop.
- **A "no-code platform."** Webflow, Framer, Wix. None of them. The whole point is that you own the code; nothing locks you into a vendor.

The tools that pay rent

By the end of this book, every dollar of that \$44 will be earning its keep. Claude Code writes. Higgsfield generates. Vercel hosts. 21st.dev inspires. Onward.

Installing and signing into Claude Code.

The single most consequential install you'll ever do. Twenty minutes.
One terminal command. One subscription. That's it.

If you've never opened a terminal before, take a breath. The terminal is just a text box. You type things into it and the computer does things. That's the whole concept. We're going to use exactly six commands in this chapter — and you can re-type them by looking at the screen.

Step one — open the terminal

On Windows, press the `Windows` key, type **Terminal**, and hit enter. On a Mac, press `⌘` + `Space`, type **Terminal**, and hit enter. You'll see a window with some text and a blinking cursor. This is where Claude Code will live.

Step two — install Node.js

Claude Code runs on a thing called Node. Go to nodejs.org and download the version it labels "LTS" (Long-Term Support). Install it. When the installer is done, close and re-open your terminal so it sees the new program, then type:

```
node --version
npm --version
```

You should see two numbers — version 18 or higher for Node, version 9 or higher for npm. If you do, you're past the hardest part.

COMMON SNAG

If your terminal says "node is not recognized" or "command not found", you forgot to close and re-open the terminal after installing. Do that, try again.

Step three — install Claude Code

One command. Paste this into your terminal and hit enter:

```
npm install -g @anthropic-ai/claude-code
```

That global flag (`-g`) tells your system to make Claude Code available everywhere — not just in one folder. If your terminal complains about permission, on a Mac put `sudo` in front of the command and type your password when asked. On Windows, run the terminal as administrator (right-click the icon → Run as administrator).

Step four — sign up for Claude Pro

Go to claude.com, click subscribe, choose Pro (\$20/mo). Use a real email — it's the same login Claude Code will use.

Step five — start Claude Code in a folder

Now we make a home for your projects. In the terminal, type:

```
cd ~  
mkdir Sites  
cd Sites  
claude
```

The first command moves you to your home folder. The second creates a folder called **Sites**. The third moves into it. The fourth starts Claude Code. It'll ask you to log in by opening a browser — say yes, hit "Authorize," and you're done. The terminal will turn into a chat box.

CHECKPOINT

If you can type "Hi" into the terminal and get a response back from Claude — you're set up. Everything else in this book is just learning what to *say*.

What you just did

You installed three things: a JavaScript runtime (Node.js), a package manager (npm, which came free with Node), and Claude Code itself. You then logged Claude Code into your Anthropic account so it knows you're a paying user.

That terminal window — with Claude waiting for input — is where you'll spend 80% of your build time from here on out.

Setting up Vercel — where your sites will live.

Vercel is the address of your website. You'll never log into a server. You'll never SSH into anything. You'll just push code to GitHub, and Vercel will turn it into a live URL while you watch.

Half the people I talk to don't understand what hosting is. They think it's a service that "stores" your website. It's not. Vercel doesn't store anything important — it **builds** your website every time you make a change, then serves the result. The thing that's stored is just the recipe; Vercel is the kitchen.

Step one — make a GitHub account

Go to github.com. Sign up. Use the same email as your Claude account if possible — fewer logins to track. Choose a username you'd be happy to see on a recruiter's screen; this username will be in URLs.

Step two — make a Vercel account

Go to vercel.com. Click "Sign up." When given the choice, sign up **with GitHub**. This binds your two accounts so Vercel can see your repos automatically. Choose the Hobby plan — it's free forever for personal projects.

Step three — install the Vercel CLI

Back in your terminal, run:

```
npm install -g vercel
vercel login
```

It opens a browser, you click yes, and your terminal is now authenticated with Vercel.

Step four — link a project

When you're inside a website's folder (we'll build one in a minute), run:

```
vercel link
```

It asks two questions: which Vercel team to use (just hit enter), and whether this is a new project (yes). A `.vercel/` folder appears in your project — that's the bookmark Vercel uses to know which project to deploy to.

How a deploy actually works

- 01 **You ask Claude to change something.**
Claude edits a file or two. You see the changes in the conversation.
- 02 **You commit and push.**
Just two terminal commands — `git add .` then `git commit -m "what I changed"` then `git push`. Claude will do these for you most of the time.
- 03 **Vercel notices.**
Within seconds, Vercel sees the new commit on GitHub. It pulls the code, runs the build, and replaces the live site.
- 04 **Your URL is updated.**
The same URL — say, `sybago.com` — now points to your new version. No outage. No "deployment in progress" page.

The whole feedback loop — change → commit → live — takes between 15 and 90 seconds. The first time it happens you'll laugh out loud.

VERCEL'S FREE TIER

The Hobby plan gives you unlimited deploys, 100 GB of bandwidth per month, and free HTTPS on every site. Every site in my portfolio runs on it. The only thing it bans is commercial use on a business you've raised money for — which, if that's where you are, you can afford the \$20 Pro plan anyway.

What you'll see in your Vercel dashboard

Once you've deployed a project, Vercel's dashboard becomes your control panel. You'll see every deploy, every preview URL, every environment variable, and a live log of every build. It is, in my experience, the single best-designed product dashboard on the internet — possibly tied with Stripe.

Setting up Higgsfield AI.

The reason my hero sections look like commercials. The fastest way to make a website feel premium without hiring a videographer.

A still photo in your hero is a website. A 6-second cinematic loop in your hero is a brand. Higgsfield is the cheapest, fastest way I've found to generate the second one.

What Higgsfield is

It's an AI video generator. You give it a prompt — or a reference image — and ten minutes later you have a 1080p video clip. The Sybago plan, \$24/month, gives you enough credits to make hundreds of clips a month, which is way more than you need.

Sign up

Go to higgsfield.ai. Sign up with email or Google. Subscribe to the cheapest paid plan (Lite, \$24/mo when I'm writing this). The free trial is real but limited; you'll burn through it on day one.

What to generate first

For your first website, generate **one hero video, one b-roll loop, and one transition clip**. That's the minimum kit. Examples from my own portfolio:

- **Hero video for a roofer** — A drone shot of a roof at golden hour, gentle pan-up, 6 seconds, looping. The site uses it autoplaying, muted, behind the headline.
- **Hero video for a luxury cologne** — Slow rotating bottle on a dark surface, lit from below, 8 seconds.
- **Hero video for a realtor** — Push-in shot through an open front door into an empty foyer.

The promptsmithing pattern I use

Higgsfield prompts work best with this skeleton:

```
[Shot type], [subject], [environment], [lighting], [motion], [duration],  
[mood], [aspect ratio], [style notes].
```

A real one I used for the JMartin Construction site:

Slow aerial drone push-forward, modern timber-frame house under construction in a Montana clearing, golden hour, soft side light, 6 seconds, hopeful and patient mood, 16:9, slight film grain, anamorphic lens flare.

Where the file goes

Higgsfield gives you a download link. I save every clip to one folder on my desktop —

`~/Desktop/HIGGSFIELD/`. From there I'll tell Claude to copy it into the project. We'll cover the wiring in Part VI.

A NOTE ON STYLE

Higgsfield **excels** at cinematic, naturalistic shots — drones, push-ins, slow pans, golden hour, architectural beauty shots. It **struggles** with fast cuts, on-screen text, hands holding objects, and faces speaking on camera. Stay in its lane.

21st.dev — the free design library.

The single best free resource for "how should this section look?"
Browse it like a lookbook. Don't copy-paste blindly. We'll cover the technique in Part V.

If Higgsfield is my photographer, 21st.dev is my mood board. It's a giant gallery of React components built by other developers. Most are free. Many are spectacular.

Sign up (optional)

You can browse 21st.dev without an account. But signing up is free and gives you a "Save" button on every component, which I use as a glorified bookmark folder. Go to 21st.dev and click sign up with GitHub.

How I actually use it

I don't sit down with a blank screen and search "hero." That's how you end up with a website that looks like everyone else's. I work backward instead:

1. Talk to the client (or the version of myself standing in for one). Get the feel.
2. Sketch the section in my head — "a sticky left column with the headline, scrolling photos on the right."
3. *Then* go to 21st.dev and find a component that's **shaped like** what I want, even if the colors and content are all wrong.
4. Read the source code. Steal the structure. Throw away the styling. Rewrite the colors and text in my brand.

What you'll find there

- Heroes (split, fullscreen, video, parallax, scroll-locked).
- Marquees that don't drop frames.
- Pricing cards with the right visual hierarchy.
- Testimonial walls and review-grid layouts.
- "Bento" feature grids — the four-or-five-tile layouts you see on Linear and Vercel's homepage.
- Animated buttons, hover-tracking pointers, magnetic CTAs.

*"Treat 21st.dev like a museum, not a Costco.
You're going for inspiration. You're not loading a
cart."*

We'll come back to 21st.dev in Part V with three line-by-line examples of how I rebuilt their components in my own sites — and why doing that, rather than importing them as-is, prevented my sites from feeling generic.



PART TWO

How any of this actually works.

Before we build anything, three short chapters on the moving parts. Once you understand the loop — Claude edits files, Git pushes them, Vercel turns them into a URL — the rest of the book stops feeling like magic and starts feeling like a recipe.

Claude Code is a junior dev who reads minds.

It's not magic. It's not even particularly clever. It's a junior developer with infinite patience, perfect recall, and access to your file system. That mental model will save you from a hundred mistakes.

The first month I used Claude Code I made every mistake you can make. I asked it questions. I argued with it. I treated it like a search engine. None of that works. Claude Code is at its best when you treat it the same way a senior engineer treats a junior — direct, clear, scoped, and quick to give feedback.

The actual loop

Every time you press enter on a prompt, Claude does the same thing in the background:

- 01 **It reads your prompt.**
Plus the surrounding conversation, plus any files it can see in the current folder.
- 02 **It plans.**
It writes — silently, to itself — a short list of what it's going to do.
- 03 **It uses tools.**
Read a file. Edit a file. Run a command in the shell. Spawn a sub-agent.
- 04 **It reports back.**
A short text response describing what it did, and the diff of any files it changed.
- 05 **You react.**
Approve, reject, redirect. The loop starts over.

What this means for how you prompt

Once you internalize "junior dev with access to my files," everything clicks. You stop asking abstract questions ("how do I make a hero?") and start giving instructions ("read `index.html`, then build a hero section above the existing content with a left-aligned headline, right-side video, and a 'Book a call' button"). Both of those prompts would technically get you a hero — but the second one gets you the *right* hero in one shot.

The three modes I work in

Every prompt I send to Claude is in one of three modes:

MODE	WHEN I USE IT	EXAMPLE
Direct	I know what I want.	"Add a testimonials section under Services. Use the three Yelp quotes in <code>copy.md</code> ."
Symptom	Something looks wrong.	"The marquee stutters every time it wraps. Kill it."
Pass	I want a whole sweep, not a fix.	"Do a mobile pass on the whole homepage. Anything cramped, fix. Anything that breaks under 380px, fix."

What Claude can't do

It can't see your screen. It can't see your design taste. It can't predict what you mean by "make it pop." So you have to translate the visual things you want into *structural* instructions:

- Instead of "make the hero feel premium" — "remove the gradient background. Use #2F6779 flat. Increase the headline to 64pt. Use 0.18em letter-spacing on the CTA button."
- Instead of "this looks generic" — "scrap the bento grid. Replace with a single 1200px-wide hero image with the copy laid on top in a left-aligned column."

The longer you work with Claude, the more you build a shared vocabulary of "small commands" — like "tighten the tracking" or "calm the gradient." Eventually it feels like talking to a colleague. But the first few weeks it'll feel like you're translating between two languages, because you are.

How Vercel turns "git push" into a live URL.

There's no magic here either. It's a small, elegant pipeline. Once you can describe it in five sentences, you'll be able to debug it when it breaks.

The first time I deployed a website I didn't really know what was happening. I just saw a green checkmark and a URL. Six months later I can describe the entire pipeline to you on a napkin, and so will you, by the end of this chapter.

The five-sentence version

You push code to a GitHub repository. Vercel watches that repository. When you push, Vercel pulls down the new code, runs a build command (something like `npm run build`), and saves the output (a folder of HTML, CSS, JS, and images). It then serves that output from a global content delivery network. Your URL — the one with `.vercel.app` in it, or your own domain — points at that CDN.

What "build" actually means

For some projects, build means "do nothing." If you have a plain `index.html` file, Vercel can serve it as-is. For others, build means "run a JavaScript bundler to turn React code into HTML." Vercel reads your `package.json` and figures out what to run automatically.

You can override the build by adding a `vercel.json` file at the top of your project. We'll get to that file in Part IX. For now, just know it exists.

Two kinds of URLs you'll get

- **Production URL** — the URL for your `master` (or `main`) branch. This is the public site.
- **Preview URL** — every other branch, every pull request, every commit, gets its own URL. So you can build something, look at it on a real URL, and only merge to master when it's ready.

UNDERUSED SUPERPOWER

Preview URLs are the single biggest reason Vercel feels different from a "real" host. You can show a client three different versions of their homepage at three different URLs, on the same day, with no extra setup.

What I check in the dashboard

1. **Deploy status.** Did the latest build succeed? Green checkmark = good.
2. **Build log.** If a deploy failed, I open the log, find the red line, and paste it back to Claude with "fix this." 90% of build failures are one missing dependency.
3. **Domains.** Once a domain is connected, this section shows DNS health.
4. **Environment variables.** Where I store API keys and other secrets. We'll cover this in Chapter 32.

What lives on Vercel that's not just static files

Every project on Vercel can have an `api/` folder. Anything in there gets turned into a serverless function — a little piece of JavaScript that runs on a server somewhere, on demand, when you hit a URL like `yoursite.com/api/lead`. We use these for contact forms, chat widgets, and review handlers. Each function gets its own URL. Each runs independently. You don't manage any servers. We'll build three of them in Part VIII.

Where your files live — the workspace anatomy.

A walking tour of my actual ClaudeCode folder. The directories matter. They're how Claude learns what kind of work it should do.

If you only remember one thing from this chapter, remember this: **Claude Code reads the folder you're sitting in.** What's in that folder shapes everything. Two files in particular — `CLAUDE.md` and the `.claude/` directory — are how you teach Claude your preferences.

The shape of my workspace

```
~/Sites/
├─ CLAUDE.md           ← global rules for this workspace
├─ .claude/
│   ├─ skills/         ← installed skills (Chapter 13)
│   ├─ settings.json   ← project-wide settings
│   └─ settings.local.json ← personal overrides + pre-approvals
├─ .git/               ← Git's bookkeeping (don't touch)
├─ sybago-website/     ← one project
├─ jmartin-construction/ ← another project
├─ coachfish-website/  ← another project
└─ ...etc
```

Every *project* is a subfolder. The folder above them is the workspace. When you run `claude` at the workspace, Claude can see all projects at once — which is how it's able to say things like "use the same chat endpoint shape as sybago-website's `api/chat.js`" when you're building the realtor site.

The CLAUDE.md file

This is the single most powerful file in the workspace. It's a markdown file at the top level that Claude reads every time you start a session. Mine looks like this (abbreviated):

```
# CLAUDE.md

## Repository purpose
This is a workspace for projects built with Claude Code.
Individual projects live as subdirectories here.

## Git workflow
- Commit after every meaningful change.
- Use clear commit messages focused on *why*, not just *what*.
- Push to origin/master after each commit.
- Always co-author commits with:
  Co-Authored-By: Claude Sonnet 4.6 <noreply@anthropic.com>

## Adding a new project
Create a subdirectory for each project. As the project takes
shape, update this file with how to run / build / test it.
```

You can write whatever you want in there. Standards. Style guides. Forbidden patterns. Claude will follow them. I tell mine to push to GitHub after every commit, and it does — every time, without me having to remember.

The .claude/ directory

This is where your customizations live. Skills (which we'll cover in Part IV), per-project settings, and a special file called `settings.local.json` where I keep pre-approvals — bash commands I always want allowed without confirming. We'll come back to all of this.

What goes inside a project folder

It varies. My static sites (sybago, jmartin, coachfish, realtor, prominent-detailing) look like this:

```
sybago-website/
├─ index.html           ← homepage
├─ pricing.html
├─ about.html
├─ styles.css
├─ main.js
├─ logo.png
├─ media/               ← videos, images
├─ api/                 ← Vercel serverless functions
│   └─ lead.js
│   └─ contact.js
└─ vercel.json          ← deploy configuration
```

My React sites (jtfasulo-website) look like this:

```
jtfasulo-website/
├─ index.html
├─ package.json
├─ vite.config.js
├─ tailwind.config.js
├─ src/
│   ├─ main.jsx
│   ├─ App.jsx
│   ├─ components/
│   │   ├─ Hero.jsx
│   │   ├─ Header.jsx
│   │   └─ Portfolio.jsx
│   │   └─ ui/          ← the shadcn-style atoms
│   └─ lib/
├─ public/              ← static assets (videos, images)
└─ vercel.json
```

You don't need to memorize either shape. Claude knows them. You'll absorb the structure by osmosis after building one or two sites.



PART THREE

The workflow.

Now we get to the part nobody else talks about. Not the tools — the moves. How I start a site. How I prompt. How I think in passes. How I commit and ship without breaking what I already had.

Starting a new site from absolute zero.

The exact sequence I use every time. Fresh folder, first prompt, Git initialized, Vercel linked, deployed before dinner.

Most beginners spend a week deciding what to build before they build anything. I've learned the opposite: build something embarrassing in an hour, then iterate. Momentum is more important than perfection in the first day.

The opening sequence

- 01 Make the folder.**
From the terminal, inside my workspace: `mkdir my-new-site && cd my-new-site && claude`.
- 02 Tell Claude what we're making.**
I open the conversation with a short brief — three to five sentences. Industry, target audience, what feeling, and one or two references.
- 03 Let Claude propose a plan.**
I ask "give me a plan before writing any code." Claude lists the sections it'll build, the tech stack, the directory layout. I correct anything off.
- 04 Build the hero first.**
Always the hero first. Nothing builds momentum like seeing your hero look right.
- 05 Init git, commit, deploy.**
As soon as the hero is acceptable — even if the rest is empty — I commit, push to GitHub, and link Vercel. Within fifteen minutes the site has a real URL.
- 06 Fill in section by section.**
Services, testimonials, FAQ, contact, footer. Each one its own conversation, its own commit, its own deploy.

The first prompt I actually send

This is verbatim from the day I started JMartin Construction:

REAL PROMPT — JMARTIN-CONSTRUCTION, DAY ONE

"I'm building a website for a custom home builder in Missoula, Montana. The client builds high-end timber-frame homes for clients who care about craftsmanship. Voice: confident, plainspoken, Montana, not flashy. Reference sites: the way Patagonia writes, the way Filson photographs.

Stack: plain HTML, hand-rolled CSS, no build step. Pages: home, six service pages (framing, siding, roofing, tile, interior-trim, decks), contact form that posts to `/api/lead.js`. Color palette: black, white, burnt orange. Big serif headlines.

Give me a plan first. Don't write code yet."

Three things to notice. **I named the industry. I named the voice. I named the visual references.** Then I named the stack, the pages, and the palette. That single message gave Claude everything it needed to build a coherent first draft.

What I don't do

- I don't ask "what should I build?" — I tell.
- I don't approve a 200-line component on the first try. I get a plan first.
- I don't worry about pixel-perfect right away. The first draft is always rough. We fix it in passes.
- I don't skip the deploy step. Even an ugly site with a real URL is more motivating than a beautiful one on my localhost.

What ships in the first hour

Realistically:

- A hero with placeholder copy (which I'll rewrite in 30 minutes).
- A two-section skeleton — services, contact.
- A Git repo initialized and pushed.
- A live Vercel URL.

The site won't look done. It'll look like a draft. That's fine. We have nine more chapters to polish it.

Thinking in passes — the mental shift that fixed my output.

Stop building "features." Build *passes*. Each pass is one concern, swept across the whole site. It sounds backwards. It's how every site I've ever finished got finished.

Early on I'd add a hero, then start tweaking it. Then I'd add a services section, and tweak *it*. By the end of day two the hero looked different from the rest of the page. Passes fix that. A pass is one concern across the whole site at once.

The passes I run on every site

- 01 **Hero pass.**
Build the hero. Just the hero. Get it to a place I'd be happy showing a friend.
- 02 **Section pass.**
Add every other section in their roughest form — services, testimonials, FAQ, contact, footer. Don't polish. Just block them out.
- 03 **Copy pass.**
Go top to bottom and rewrite every headline, subhead, and body block. This is where I find the voice. Sometimes I write copy in a separate `copy.md` file first and feed it to Claude.
- 04 **Design pass.**
Spacing, typography scale, alignment, color consistency. "Make every section have 6.5rem of vertical padding. Make every h2 the same size. Tighten the headline tracking to -0.02em."
- 05 **Mobile pass.**
Open Chrome DevTools, switch to mobile, and walk every section. Anything that breaks, fix. Anything that's cramped, breathe out.
- 06 **Perf pass.**
Lighthouse score. Lazy-load any below-the-fold video. Compress hero images. Move heavy animations off the main thread.
- 07 **SEO pass.**
Title tag, meta description, OG image, JSON-LD, sitemap, canonical, theme-color. Maybe

twenty minutes total.

08

Final polish pass.

Hover states, focus rings, error states, empty states, 404 page. The little things that make a site feel inhabited.

Why passes work and feature-by-feature doesn't

When you build feature-by-feature, the decisions you made on feature one bleed into feature two before you've decided what features three through eight need. By the time you're at the bottom of the page, the top doesn't match. Passes prevent this by forcing one concern at a time across the whole surface area.

How I phrase a pass to Claude

REAL PROMPT — MOBILE PASS ON REALTOR-WEBSITE

"Mobile pass on the whole homepage. Open `index.html`. For every section, check the breakpoints at 380px, 480px, and 768px. Anything that overflows horizontally, fix. Anything where text is touching an image edge, add padding. Anything where a 2-column layout is cramped under 768px, drop to one column. Don't change desktop. Don't change copy. Just mobile."

Notice the constraint stack at the end — "don't change desktop, don't change copy" — that's a guardrail. Claude will sometimes "improve" things you didn't ask it to. Naming the boundary keeps it inside the pass.

The language tricks that actually move Claude.

After hundreds of hours in the terminal I've narrowed down what gets results to a small set of phrasings. Here they all are, with examples.

Most prompting advice on the internet is generic. "Be clear. Be specific." Sure. Here are the specific phrasings I lean on — the actual words that bend the output in the direction I want.

1. Name the symptom and the feel

Not "make the marquee better." Not "fix the marquee." Instead: *"The marquee stutters every time it wraps. Kill the stutter — should feel like a continuous river of logos."*

Symptom (stutters at wrap) + desired feel (continuous river). Claude knows what to do.

2. Anchor to an existing file

If you've already done something similar in another project, reference it. *"Use the same chat endpoint shape as `sybago-website/api/chat.js` — POST { messages: [{role, content}] } → { reply: string }."*

You preserve your own conventions across projects, and Claude doesn't reinvent the wheel each time.

3. Constrain the change

When you're doing a focused tweak, name the boundaries. *"Just the hero. Don't touch anything else."* Or *"CSS only. No JavaScript."* Or *"Don't change the copy."* Without guardrails, Claude will sometimes "improve" the thing you weren't asking about.

4. Ask for a plan before code

On anything bigger than a single tweak: *"Give me a plan first. Don't write code yet."* You'll get a numbered list. Approve it, or send corrections. Saves a hundred wasted minutes.

5. Use "pass" as a verb

It signals scope. *"Mobile pass on the homepage."* *"Copy pass on the services section."* Claude has learned (from millions of code-review threads in its training set) that "pass" means "sweep, one concern at a time."

6. Name a font, a number, a hex

Specific beats vague every time. Not "make the headline bigger." Instead: *"Bump the hero h1 to clamp(2.4rem, 5.8vw, 4.4rem). Tighten letter-spacing to -0.02em. Set line-height to 1.04."*

7. Quote the customer

When writing copy, give Claude actual phrases the customer says. *"My client describes his work as 'crafting timber-frame homes the way our grandfathers did.' Use that phrasing in the hero. Don't make it cute."*

8. Forbid common defaults

Claude has favorite patterns that will make your site look generic. Block them explicitly. *"Don't use Inter. Don't use Space Grotesk. Don't use a purple-to-blue gradient anywhere. Don't use shadcn's default card."*

"Specific beats clever. Numbers beat adjectives. References to your own past work beat references to 'best practices.'"

9. Reward what works, redirect what doesn't

Claude listens. If a particular choice landed, say so. *"The em-dash separator on the numbered labels is perfect. Use that pattern on the pricing table too."* If something didn't, redirect specifically. *"The badge above the headline feels like a startup template. Replace with a plain eyebrow — uppercase, 0.32em tracking, no background."*

10. Talk like you're talking to a colleague

Imperative voice. Active verbs. Short sentences. *"Pull the hero out. Drop in the new one. Wire the CTA to `/checkout`."* Not *"Could you please be so kind as to..."* — that's not politeness, that's noise.

Commit early. Commit often. Push every time.

My single most important habit. The reason I've never lost a day's work. The reason every site has a clean history I can read back like a diary.

Git is the safety net under everything I do. I commit after every meaningful change — even if "meaningful" means a single hero tweak. I push to GitHub every time. The result: at any moment, I can roll back to any version of any site I've ever built, on any machine, in fifteen seconds.

The rules I follow

- **Commit after every meaningful change.** Not every keystroke. Every meaningful change — a new section, a fixed bug, a redesigned hero. If I'd be sad to lose it, I commit.
- **Write commits that explain *why*, not *what*.** `fix typo` is useless. `Tighten hero tracking – felt loose against the new wider container` tells future-me what I was trying to do.
- **Push every commit.** Always to `origin/master`. So that if my laptop falls in a lake, I can rent a new one and pick up exactly where I left off.
- **Project prefix on every commit.** `sybago: tighten hero h1 tracking`. `jmartin: fix marquee stutter`. So when I scroll my history I can tell at a glance which site I was on.

Real commit messages from my workspace

```
jmartin: smooth out testimonials marquee – kill stutter at the wrap
jmartin: center services carousel cards when they fit
cologne: trim carousel to 5 cards + continuous fractional scroll
realtor: wire chat widget to Claude Haiku backend
sybago: rebuild pricing page with money-back guarantee chip
Canvas-render CTA paths; gate reviews animation until measured
Replace Tailwind Play CDN with precompiled stylesheet
Tune liquid-metal hero to flow with chaotic fluid mechanics
```

Each one tells me: what project, what changed, and (often) why. Months later, I can read back through and remember the decisions.

How Claude helps with commits

If you've put the right instructions in `CLAUDE.md`, Claude will commit for you. Mine says "commit after every meaningful change. Push to origin/master." So after a non-trivial change I just say "commit and push." Claude runs `git add`, drafts a commit message, asks me to approve, and pushes. The whole thing takes ten seconds.

PRE-APPROVING THE PUSH

In `.claude/settings.local.json`, you can pre-approve specific bash commands so Claude doesn't ask permission each time. I've allowed `git push`, `git status`, `git diff`, and a handful of other read-only operations. Result: my flow never breaks for permission prompts.

What I do when something breaks

Always, the first move: `git log --oneline -10` in the terminal. Find the last commit where it worked. Then either:

- Revert just that one commit: `git revert <commit-hash>`.
- Or roll the whole working copy back: `git checkout <commit-hash> -- .` (careful — this wipes uncommitted changes).

The point is: I have options because I committed often. The people I see panicking lost work either committed once a day, or never.



PART FOUR

Skills. The unfair advantage.

If Claude Code is the worker, skills are the specialized training manuals you hand it. Install once, and Claude knows how to do something a particular way, forever. This is where the gap between "AI website" and "premium website" closes.

What skills are, and why I install one per problem.

A skill is a folder. Inside the folder is a markdown file. The markdown file tells Claude how to do something — when, why, and what rules to follow. That's it. That's the whole technology.

When I first heard about skills I assumed they were a complex piece of infrastructure. They're not. A skill is a markdown file in a specific folder. Claude reads it when relevant and follows the instructions inside. The reason they're powerful is not the format — it's that they encode the lessons you've already learned.

Anatomy of a skill

```
~/Sites/.claude/skills/animation-performance/  
└─ SKILL.md          ← the actual content
```

Inside `SKILL.md` :

```
---  
name: animation-performance  
description: Apply when building scroll-scrubbed animations,  
  marquees, continuous SVG animations, or fixing reports of  
  "laggy", "stuttering", "doesn't render".  
---  
  
# Animation performance — the ten rules  
  
1. Long SVG dash-offset animations belong in <canvas>, not SVG.  
2. Marquees run on CSS @keyframes, not requestAnimationFrame.  
3. ...
```

The frontmatter tells Claude *when* to use the skill. The body is the lesson.

Why I install one per problem

Every site I build teaches me something. Marquees lag a certain way on Safari. Frontend "creative" defaults look generic on a contractor site. AI video has specific rules for what reads well. Each lesson, if I don't capture it, evaporates. So I write it down — once — and now every future site gets the benefit

of the lesson without me having to remember.

The animation-performance skill exists because the first version of `realtor-website` had laggy SVG. I fixed it. I wrote down what I learned. Now every site I build gets fast animations on the first try.

The skills I have installed (and why)

SKILL	WHAT IT DOES	WHY I INSTALLED IT
frontend-design	Pushes for bold, distinctive design — bans Inter, Roboto, generic gradients.	Stops Claude defaulting to AI-slop aesthetic.
animation-performance	10 rules for smooth marquees, canvas-vs-SVG, gating animations until measured.	Eliminates lag I learned to spot on the realtor site.
branded-carousel	One-shot Instagram carousels in my brand. Locks colors, fonts, slide architecture.	So I can ship a carousel in two minutes when I need one.
ai-creator-video-style	Editing recipes for Reels/Shorts in Remotion, in the styles I've reverse-engineered.	For the short-form video that goes alongside the site.
remotion-best-practices	Rules for building Remotion compositions correctly.	Stops Claude from making structurally invalid videos.
frontend-design / browser	Browser automation for screenshot-comparing my own work.	To verify mobile + desktop look right.

I'll spend the next three chapters on the most important ones.

frontend-design — escaping generic AI slop.

The single most consequential skill in my install. The reason my sites look like sites people made, not sites a robot generated.

Out of the box, Claude has favorite default choices. Inter as the font. A purple-to-blue gradient on the hero. Three pricing tiers in shadow cards with the middle one highlighted. A bento grid for features. Round corners on everything. You've seen the site Claude builds when you don't push it — there are thousands of them on Twitter every day.

What the skill does

The frontend-design skill tells Claude to **avoid** those defaults. To pick a typography pair you've actually never seen. To use one dominant color with one sharp accent — not a gradient. To use noise textures, gradient meshes, asymmetric layouts. The skill is mostly a list of bans.

What the rules actually say

- **Don't use Inter, Roboto, or Arial.** Pick a font with character. Garamond. Suisse. Fraunces. Cormorant. Editorial New.
- **Don't converge on common choices.** Space Grotesk is banned by name. So are the obvious shadow defaults.
- **Use CSS variables for colors and spacing.** Centralized. Themeable. Override-able later.
- **One dominant color + sharp accents.** Not three colors fighting. Sybago is teal-on-white with one green chip; that's the model.
- **Gradient meshes are okay; gradient text and gradient buttons are not.**
- **Noise textures are encouraged.** A subtle film grain on a flat background reads as premium.
- **Type should be the design.** The size, weight, and tracking of your headlines is the visual identity — not the icons next to them.

How I trigger it

The skill auto-fires on phrases like "build a hero," "redesign the homepage," "I want this to feel more premium." But I also call it explicitly: *"Use the frontend-design skill. I don't want a generic startup look."*

THE BIGGEST SINGLE TELL

If your site uses Inter and a purple-to-blue gradient, it looks like every other AI-generated site. Pick a different font and a different color. That one change does 60% of the work.

What it looks like in practice

Sybago: Inter — yes, but in 800 weight at 4rem with -0.02em tracking, against #2F6779 teal, with everything else hairlines and whitespace. Not generic. JMartin: Fraunces for display, the rest of the site is restraint. Coachfish: Unbounded + Montserrat. Cologne: a single serif at 96pt over a black background. Each one of those choices came out of the frontend-design skill telling Claude "no Inter, no purple-blue, no shadcn." Then I picked the alternative.

animation-performance — the ten rules.

Every rule in this skill was learned by watching my own site stutter. They're not theory. Each one fixes a specific lag I once had on a real project.

Animation is where most "AI-built" sites fall apart. The first version looks great in a screenshot, then you scroll and it jitters every wrap. The skill encodes the ten fixes I learned the hard way.

The ten rules

- 01 Long SVG dash-offset animations belong in <canvas>.**
SVG path animation forces a re-layout every frame on most browsers. Rewriting the same animation in canvas gets you 60 fps for free.
- 02 Marquees run on CSS @keyframes, not requestAnimationFrame.**
RAF runs on the main thread. CSS animations run on the compositor. The compositor is always available; the main thread is fighting React.
- 03 mask-image is forbidden on transforming children.**
Combining a mask with a translate is a known browser slow-path. Replace with a plain overflow:hidden parent.
- 04 Triple-buffer at scrollHeight / 3.**
For seamless infinite scroll loops, render three copies of your content and snap back at the 1/3 mark. Anything else jumps visibly.
- 05 Gate animations until measured.**
Use IntersectionObserver and don't start until the element is on screen. Don't waste cycles animating offscreen content.
- 06 No box-shadow on transforming elements.**
Big shadows are the single biggest paint cost in the browser. If something's moving, shadows go.
- 07 No backdrop-filter on transforming elements.**

Same reason — composites everything every frame.

08

vector-effect="non-scaling-stroke" on every animated SVG path.

Without it, browsers redraw the stroke at every transform.

09

Will-change is a hint, not a fix.

Use it sparingly. It pins the layer to GPU memory; overuse evicts everything else.

10

Test on the slowest device you can find.

If it's smooth on a \$200 Android, it's smooth everywhere.

Where the rules came from

The realtor-website hero used a beautiful SVG dash animation of a roof outline. Looked great. Couldn't scroll past it without the whole site dropping to 12 fps. I rewrote it in canvas; back to 60 fps. That fix became rule one.

The same site had a testimonials marquee that stuttered on Safari. I'd been using `requestAnimationFrame` with `translateX`. Replaced with a CSS `@keyframes` animation. Stutter gone. That fix became rule two.

And on, and on. The skill is a journal of every animation that ever broke on me, with the fix attached.

"Performance bugs are skill-shaped. Every one I fix becomes a rule I never write again."

The other skills I use weekly.

Quick tour of the rest. branded-carousel for Instagram. ai-creator-video-style for Reels/Shorts. Remotion best practices for the heavy lifting.

branded-carousel

One-prompt Instagram carousels in my brand. The skill knows my colors (the Sybago teal, black, white), my font (Fraunces for the cover, Inter for everything else), my slide sizes (1080×1350), and my seven-slide architecture (Cover / Context / Three value slides / CTA). I say "make me a carousel about [topic]" and it produces seven slides.

The skill defaults to building via the Canva MCP — sending direct API calls into Canva's design engine — so the slides come back as a real Canva document I can tweak. There's a Nano Banana fallback for when Canva's API hiccups.

ai-creator-video-style

Reverse-engineered editing recipes for the kind of short-form video that goes alongside my websites. Two sub-styles — "Daylight Apartment" (warm, soft) and "Dark Studio" (high-contrast, cinematic). The skill encodes the exact frame rate (23.976), the exact caption typography (Inter 800 with a soft text-shadow), the exact way to overlay iMessage bubbles. I produce videos by giving the skill the source clip and the script.

remotion-best-practices

Remotion is React-based video composition — write JSX, render frames, get an MP4 out. It's how I produced the Sybago feature animations and ad reels. The skill keeps Claude from making structurally invalid Remotion compositions (mistakes that compile but render to a static frame).

The plugin-author skills

I also have skills for building skills — agent-development, command-development, hook-development, plugin-structure, skill-development. You only need these if you start building your own. Don't worry about them for now.

How to write your own skill

When you've solved the same problem twice, write the third solution into a skill. Mine usually start with five lines of frontmatter (the name and the trigger), and grow into a markdown file of 20–80 lines of rules. The skill-development skill itself walks you through the structure.

```
~/Sites/.claude/skills/my-new-skill/  
└─ SKILL.md  
    └─ frontmatter: name + description (when to fire)  
        └─ body: rules, examples, gotchas
```


PART FIVE

Designing with 21st.dev.

Three chapters on how I use 21st.dev — not as a copy-paste shortcut, but as a lookbook. Why I rewrite every imported component by hand. And three real examples of components I rebuilt from scratch to keep my site fast.

How I browse 21st.dev for inspiration.

Not a Pinterest scroll. A focused half-hour with a specific brief. Walk away with three ideas, not thirty.

When I sit down with 21st.dev, I have one job: find a structural pattern. Not a color palette. Not a final design. A structural pattern — a way of arranging content I haven't seen on my site before.

My browsing routine

- 01 **Decide what section I'm browsing for.**
"Hero." "Testimonials." "Feature grid." Specific.
- 02 **Set a 25-minute timer.**
Browsing is procrastination. The timer keeps it from becoming the whole day.
- 03 **Open three tabs of components I like.**
Three. Not thirty.
- 04 **Read the source code of each.**
What's the structure? What's the layout grid? What's the JavaScript doing? How are they animating?
- 05 **Pick one.**
The one with the structure closest to what I want — even if the colors are wrong.
- 06 **Close the tab. Go build.**
I never copy the source verbatim. I describe the structure to Claude and have it rebuild in my brand.

The categories I use most

- **Heroes** — for the opening section pattern (split, fullscreen, video, parallax, scroll-locked).
- **Bento grids** — for feature sections.
- **Marquees** — for logo strips and testimonial scrollers.
- **Pricing** — for the pricing page, when the design needs a fresh angle.

- **Buttons** — magnetic, hover-tracking, glow, all of it.
- **Cards** — testimonial cards, feature cards, blog cards.

What I skip

Most "AI-themed" components. Most "dashboard" components. Anything that looks like it was built for a SaaS startup that hasn't shipped yet. The aesthetic I'm building isn't dashboard-y; it's editorial.

Why I rewrite every imported component by hand.

Copy-pasting a component from 21st.dev installs three problems at once: dependencies you don't need, a visual style that isn't yours, and animations on the wrong thread. Rebuilding takes the same amount of time and avoids all three.

The first time I imported a 21st.dev component I felt clever. I had a beautiful PerspectiveMarquee in my site within two minutes. I felt less clever the next morning when I noticed my page now imported the entire Remotion runtime and ran a JavaScript animation loop every frame.

The three problems with raw imports

- **Dependency creep.** Most 21st components are built in someone else's stack. Importing one component often drags in two or three npm packages — which slow your build, bloat your bundle, and clutter your `package.json`.
- **Style collision.** The component looks great on the 21st.dev demo because of styles you can't see. Drop it into your site, and it falls apart against your real backgrounds.
- **Animation on the main thread.** A lot of components use `requestAnimationFrame` to drive their motion — beautiful on a demo page that has nothing else running. Catastrophic on a real homepage with React reconciling.

What I do instead

I read the component. I understand the structure. I tell Claude what the structure is, in plain English, and ask Claude to rebuild it in my stack, in my brand.

A REAL INSTRUCTION I SENT

"Look at the 21st.dev PerspectiveMarquee component. The structure: a row of images, scrolled horizontally on a continuous loop, but with each image rotated slightly in 3D space so the row feels like it curves away. Rebuild it for jtfasulo-website. Plain React. No Remotion. Drive the animation with CSS `@keyframes` — no `requestAnimationFrame`. Use my five client logos from `public/clients/`."

The result was 50 lines of clean JSX, no external dependencies, and a smooth 60 fps animation on the compositor thread. Took the same time as a copy-paste would have, and didn't break anything.

When I do import directly

Rare. Only if the component is genuinely self-contained — no external dependencies, no RAF loop, no problematic styling. The shadcn primitives qualify. Most 21st components don't.

Three real examples — marquee, hover-button, zoom tile.

Three components I found on 21st.dev, rewrote by hand, and shipped to production. The before, the after, and the reason.

Example 1 — PerspectiveMarquee

Source: a 21st.dev component that uses Remotion's `useCurrentFrame()` to drive a `translateX` every render.

Problem: Remotion is a video framework. Importing it for a marquee installs a 200kb dependency and runs your animation on the main thread.

Fix: Rebuilt as plain JSX. Animation is a single `@keyframes` rule that translates a duplicated row of logos -50% over 30 seconds.

```
@keyframes marquee {  
  from { transform: translateX(0); }  
  to   { transform: translateX(-50%); }  
}  
  
.row {  
  display: flex;  
  width: max-content;  
  animation: marquee 30s linear infinite;  
}
```

The duplication trick (rendering the row twice in a flex container and translating -50%) gives you a seamless loop with zero JavaScript.

Example 2 — HoverButton

Source: a 21st button that tracks the cursor inside its bounds and emits short-lived gradient blur dots that fade in then out.

Problem: The original used CSS-in-JS and tracked the cursor via React state — re-rendering the button on every `mousemove`. On mobile, useless; on desktop, expensive.

Fix: Rebuilt with vanilla CSS variables. I track the cursor with one `mousemove` listener, write x/y to CSS custom properties on the button, and let CSS do the rest — no React re-render. Same look, no state churn.


```
btn.addEventListener('mousemove', (e) => {  
  const r = btn.getBoundingClientRect();  
  btn.style.setProperty('--mx', `${e.clientX - r.left}px`);  
  btn.style.setProperty('--my', `${e.clientY - r.top}px`);  
});
```

Example 3 — Zoom-Parallax Tile

Source: a 21st component that pins a central tile and zooms it as you scroll, with eight surrounding tiles scaling down.

Problem: The original used a scale ramp `[4, 5, 6, 5, 6, 8, 9]` driven by JavaScript on every scroll event.

Fix: Same scale ramp, but expressed as keyframes tied to scroll position via the View Transitions API. Smooth, GPU-driven, no scroll listener.

"The component is the recipe. Your kitchen is your stack. Cook it your way."

PART SIX

Higgsfield AI for scroll animations.

Four chapters on the secret weapon that makes my sites feel like commercials. What to generate. How to prompt. How to wire it into your site without killing performance. And the file-size limit you'll absolutely hit.

What Higgsfield is great at — and what it isn't.

Higgsfield is a precision instrument. Used in its lane, it produces footage your client will assume came from a film crew. Used outside its lane, it produces uncanny garbage.

I've spent more time on Higgsfield than any other AI tool in my stack. Most of that time, in the early days, was learning what it can't do. Save yourself the months — here's the map.

Where it excels

- **Cinematic establishing shots.** Drone push-ins, dolly-pans, slow architectural pulls. Anything that imitates a film camera doing one tasteful thing.
- **Texture and material.** Light hitting timber. Water beading on glass. Smoke curling under stage lighting. The macro-aesthetic stuff.
- **Atmosphere.** Golden hour, blue hour, foggy morning, neon night. Higgsfield's lighting models are excellent.
- **Slow motion.** 4–8 seconds is the sweet spot. Anything you can do in 8 seconds, you can do well.
- **Image-to-video.** Give it a still photo of a finished kitchen and ask for a slow push-in. Better than starting from a text prompt.

Where it struggles

- **People speaking on camera.** Lip sync is unconvincing. Avoid.
- **Hands holding objects.** The number of fingers is unpredictable. The way an object is held is awkward. Don't show hands.
- **Fast cuts.** Higgsfield outputs single continuous shots. Cuts have to be done in post.
- **On-screen text.** Don't bother. Generate the shot without text and overlay your own type later.
- **Brand-accurate logos.** Won't reproduce a real logo precisely. Composite your logo on top.

The rule I follow

If a scene needs a person to be the subject, I'll either skip AI video for that scene (use real footage, or stay on a still) or use Higgsfield for the establishing shot and cut to real footage of the person. AI is the atmosphere; people are the substance.

SAMPLE BRIEF — JMARTIN CONSTRUCTION SITE

I generated three clips total. (1) Drone push-forward over a timber-frame house under construction at golden hour. (2) Slow pan across a finished kitchen interior. (3) Vertical drone pull-back from a roofline. Each clip was 6 seconds. Each took 8–12 minutes to render. The whole video kit for the site cost about \$4 of credits.

My exact Higgsfield prompt patterns.

The skeleton I use for every clip. Real examples. The four edits that move quality the most.

Every Higgsfield prompt I write follows the same skeleton. I started using it after my third "the shot was great but the camera moved the wrong way" experience. Specifying camera motion explicitly fixes most directability problems.

The skeleton

[CAMERA MOTION], [SUBJECT], [LOCATION], [LIGHTING],
[DURATION], [MOOD], [ASPECT RATIO], [STYLE NOTES].

Each field is one phrase. Eight short phrases beats two long ones every time.

Real prompts I've used

REALTOR HERO

"Slow forward push, through an open front door into a sunlit empty foyer, modern minimalist home, late-afternoon golden hour, soft directional sunlight, 6 seconds, calm and inviting mood, 16:9, slight film grain, shallow depth of field."

COLOGNE HERO

"Slow 360-degree rotation, amber glass perfume bottle on a dark stone surface, dimly-lit cinematic studio, single soft key light from above, 8 seconds, mysterious and luxurious mood, 16:9, anamorphic lens flare on highlights."

JMARTIN CONSTRUCTION HERO

"Slow drone push-forward, modern timber-frame house under construction in a Montana mountain clearing, late-day golden hour, soft side lighting, 6 seconds, hopeful and patient mood, 16:9, slight film grain, anamorphic lens flare."

The four edits that move quality the most

01

Be specific about camera motion.

"Slow push" beats "movement." "Aerial pull-back" beats "drone." Verbs over nouns.

02

Always specify lighting.

Golden hour. Blue hour. Soft side light. Hard top light. Lighting alone differentiates a \$4 clip from a \$400 one.

03

Constrain the duration.

I always say 5, 6, or 8 seconds. Higgsfield handles those lengths best.

04

Add one signature lens detail.

"Slight film grain." "Anamorphic lens flare." "Shallow depth of field." These nudge Higgsfield toward cinematic, away from stock.

What I never do

- Use the word "professional." Means nothing.
- Use the word "stunning." Means nothing.
- Ask for two camera movements in one prompt (it'll pick one and ignore the other).
- Specify a person's face or expression in detail (Higgsfield is loose about faces).

Wiring the video into a hero — without killing Lighthouse.

A video hero done wrong tanks your performance score. Done right, it costs you almost nothing. The difference is six lines of HTML.

A 4 MB video file looks great. It also delays your page load by two seconds, which Google notices, which Lighthouse punishes, which costs you SEO. The fix isn't to skip the video — it's to load it properly.

The HTML pattern I use

```
<video
  class="hero-video"
  autoplay
  muted
  playsinline
  loop
  preload="metadata"
  poster="/hero-poster.jpg">
  <source src="/hero.webm" type="video/webm">
  <source src="/hero.mp4" type="video/mp4">
</video>
```

Six lines. Each one matters.

- **autoplay muted playsinline loop** — the four attributes browsers require to autoplay video without user interaction. Skip any one and Safari won't play it.
- **preload="metadata"** — load only the first frame and the duration, not the whole file. The poster image takes over visually while the rest streams in.
- **poster="/hero-poster.jpg"** — a still frame, usually 100kb, shown while the video downloads. *This is the single most important attribute for performance.*
- **Two source files** — WebM first (smaller, better quality per kb), MP4 fallback (Safari users).

Generating the poster

I pull a single frame from the video using ffmpeg, downscale it to 1920×1080, save as JPG with 80% quality. Result: ~80–120kb. Looks identical to the first frame of the video. Loads instantly.

```
ffmpeg -i hero.mp4 -ss 00:00:00 -vframes 1 -q:v 2 hero-poster.jpg
```

Where the videos actually live

For static sites: directly in the project under `media/` or `public/videos/`. Vercel serves them off its CDN.

For larger files (over 5 MB): I've started using Cloudflare R2 or Bunny CDN. Cheap, fast, and keeps your Vercel deploy size down.

VERCEL LIMIT YOU'LL HIT

Vercel's free plan caps deploys at 100 MB of static assets. If your hero video plus thumbnails plus a logo set crosses that, the deploy fails. Either compress more aggressively (next chapter), or move heavy files to an external CDN.

Compressing, hosting, and the limit you'll hit.

The numbers I aim for. The ffmpeg commands I run. The CDN I move to when the free plan runs out.

Higgsfield exports a 1080p MP4 at ~10 MB per 6 seconds. That's too big for the web. Compressed correctly, the same clip can be 1.2 MB and indistinguishable. Here's exactly how.

My target numbers

ASSET	TARGET SIZE	ACCEPTABLE MAX
Hero MP4 (6 seconds, 1080p, muted)	1.2 MB	3 MB
Hero WebM (same)	800 KB	2 MB
Hero poster JPG	80 KB	150 KB
Background looping b-roll	500 KB	1.5 MB

The ffmpeg commands

Two commands. Run them on every Higgsfield export.

```
# MP4 (H.264) — ~1.2 MB per 6 seconds at 1080p
ffmpeg -i input.mp4 -c:v libx264 -crf 28 -preset slow \
  -movflags +faststart -an output.mp4

# WebM (VP9) — ~800 KB per 6 seconds
ffmpeg -i input.mp4 -c:v libvpx-vp9 -crf 34 -b:v 0 \
  -an output.webm
```

The flags that matter:

- **-crf 28 (MP4) or 34 (WebM)** — quality. Lower = bigger file. 28 / 34 is my sweet spot for hero video.
- **-preset slow** — let the encoder think harder. Better compression for the same quality.
- **-movflags +faststart** — puts the video header at the start of the file so the browser can start playing before the whole thing has downloaded.

- **-an** — strip the audio track. Hero videos are always muted; the audio is just dead weight.

Where I host once I outgrow Vercel

- **Cloudflare R2** — S3-compatible, no egress fees, ~\$0.015/GB/month. My default once a site has more than 50 MB of video.
- **Bunny CDN** — paid CDN with a \$1/month minimum. Excellent for video. Used it for the cologne-website's 605-frame scroll sequence.
- **Backblaze B2** — cheapest of all. I'd pair it with a Cloudflare worker as a CDN front-end.

Don't pre-optimize. Start on Vercel. When you outgrow it, you'll know — your deploy will fail and the error message will tell you.

PART SEVEN

SEO that actually works.

Five chapters on the unglamorous fundamentals. Title tags. Meta descriptions. Open Graph. JSON-LD schema. The one weird h1 trick I use on every contractor site. Sitemaps. Robots. Done in twenty minutes, paying you back for years.

The title-tag pattern that ranks contractor sites.

Sixty characters. Three jobs. One template you can paste over every site you build.

Of every SEO lever you'll touch, the title tag is the biggest one. It's the line that shows up in Google. It's the line Google ranks. It's the one most people get wrong by trying to be clever.

The template

```
<title>[Service] in [City], [State] | [Brand]</title>
```

That's it. Three jobs. Service first (what they search). City next (where they search). Brand last (so Google ties it to your domain). Sixty characters or fewer (longer gets truncated in the SERP).

Real examples from my sites

```
<title>Custom Home Builder in Missoula, MT | JMartin Construction</title>
<title>Auto Detailing in Phoenix, AZ | Prominent Detailing</title>
<title>Pressure Washing & Window Cleaning | Prominent Cleaning Co.</title>
<title>Sybago – Stop losing leads to faster contractors. $297/mo.</title>
```

For the trades sites: service + city + brand. For Sybago (a product, not a local business): the value prop + the price, because a search like "missed call text back software pricing" is the most likely route in.

One title per page

Every page gets its own title tag. The homepage gets the headline service. Each service page gets *that* service plus the city. So for JMartin:

```
/ → Custom Home Builder in Missoula, MT | JMartin
/services/framing → Timber-Frame Framing in Missoula | JMartin
/services/roofing → Roofing in Missoula, MT | JMartin
/services/siding → Siding Installation in Missoula | JMartin
```

Google indexes these as separate landing pages, each ranking for its own specific search.

What I never put in a title

- The word "Best." Adds nothing, and Google ignores adjectives.
- The word "Welcome." Burns characters.
- A tagline that doesn't include a keyword.
- Pipe-separated lists of unrelated services. Pick the one that matters most.

Meta description, Open Graph, Twitter card.

The three other meta tags that matter. Together with the title, they're 90% of how your site appears anywhere it's linked.

Meta description

Roughly 150 characters. Pitch the product. Include the city. Include the price if you have one.

```
<meta name="description" content="Custom timber-frame home builder  
serving Missoula and Western Montana since 1998. Built like our  
grandfathers did, finished like the future. Free quote in 24 hours.">
```

Open Graph (for Facebook, LinkedIn, iMessage previews)

```
<meta property="og:title"          content="Custom Home Builder in Missoula">  
<meta property="og:description"    content="Timber-frame craftsmanship...">  
<meta property="og:image"         content="https://yoursite.com/og.jpg">  
<meta property="og:type"          content="website">  
<meta property="og:url"           content="https://yoursite.com">
```

Twitter card

```
<meta name="twitter:card"          content="summary_large_image">  
<meta name="twitter:title"         content="Custom Home Builder in Missoula">  
<meta name="twitter:description"    content="Timber-frame craftsmanship...">  
<meta name="twitter:image"         content="https://yoursite.com/og.jpg">
```

The OG image — what to make

1200 × 630 pixels. JPG, ~150 KB. I make mine in Figma or by exporting a poster frame from the hero video. Include the brand wordmark, the headline, and a phone number or CTA. This image is what people see in iMessage, Slack, Twitter, LinkedIn — anywhere the site is shared. It's the second-most-important visual asset on the site after the hero itself.

A QUICK TEST

After deploying, paste your URL into `opengraph.xyz`. You'll see exactly how the link unfurls on every major platform. Fix anything that doesn't look right.

JSON-LD schema — the structured data Google reads.

A block of JSON in your <head> tells Google what kind of business you are, where you are, what you sell, and when you're open. Copy-paste templates for the three types I use most.

JSON-LD is the most boring chapter in this book. It's also the one with the highest ROI per minute spent. Twenty minutes of structured data unlocks rich results in Google — star ratings, business cards, opening hours — that can quadruple your click-through rate.

Template 1 — Organization (for SaaS, product, or services company)

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Organization",
  "name": "Sybago",
  "url": "https://sybago.com",
  "logo": "https://sybago.com/logo.png",
  "description": "Lead capture and review automation for trades.",
  "sameAs": [
    "https://www.linkedin.com/company/sybago",
    "https://twitter.com/sybagohq"
  ]
}
</script>
```


Template 2 — LocalBusiness / GeneralContractor (for trades sites)

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "GeneralContractor",
  "name": "JMartin Construction",
  "image": "https://jmartin.com/og.jpg",
  "address": {
    "@type": "PostalAddress",
    "streetAddress": "1234 Reserve St",
    "addressLocality": "Missoula",
    "addressRegion": "MT",
    "postalCode": "59801",
    "addressCountry": "US"
  },
  "telephone": "+1-406-555-0100",
  "url": "https://jmartin.com",
  "priceRange": "$$$",
  "openingHours": "Mo,Tu,We,Th,Fr 08:00-17:00",
  "areaServed": [
    { "@type": "City", "name": "Missoula" },
    { "@type": "City", "name": "Hamilton" },
    { "@type": "AdministrativeArea", "name": "Western Montana" }
  ],
  "knowsAbout": ["Timber Frame", "Custom Homes", "Roofing", "Siding"]
}
</script>
```

Template 3 — Product (for a paid PDF or digital good)

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": "Build $10k Websites With Claude Code",
  "description": "A 40+ page playbook for building production websites...",
  "brand": { "@type": "Brand", "name": "Sybago" },
  "offers": {
    "@type": "Offer",
    "priceCurrency": "USD",
    "price": "19.99",
    "availability": "https://schema.org/InStock"
  }
}
</script>
```

How to verify it works

Paste your URL into `search.google.com/test/rich-results`. Google will tell you exactly what it found and what's missing. Fix until green.

The hidden-h1 / visual-h1 split.

One trick. Two h1s. One is for Google. The other is for humans. Both happy.

A traditional SEO h1 is a benefit-led, keyword-stuffed sentence: "Custom Residential Home Builder in Missoula and Western Montana." A traditional marketing h1 is a punchline: "Built like grandfather did. Finished like the future." Most sites pick one and lose half the value. I do both.

The pattern

```
<h1 class="seo-h1">
  Custom Residential Home Builder in Missoula and Western Montana
</h1>

<p class="hero-display">
  We build the home you've been drawing in your head.
</p>
```

The `seo-h1` class is the keyword h1 — but I hide it with CSS so it doesn't dominate the design. The `hero-display` p is what the visitor actually reads.

The CSS that hides the SEO h1 — without hurting accessibility

```
.seo-h1 {
  position: absolute;
  left: -10000px;
  top: auto;
  width: 1px;
  height: 1px;
  overflow: hidden;
}
```

This is the "visually hidden" pattern used by every screen-reader-friendly site. The h1 is still in the document, still read by Google's crawler, still read by screen readers — just not visually rendered.

Why this works

Google reads the h1 for ranking. Humans read the visual headline for engagement. By splitting them, you optimize both audiences without compromise. I do this on every trades site I build.

IMPORTANT CAVEAT

This is not cloaking. Cloaking is showing different content to Google than to humans for the purpose of deceiving the ranker. This is presenting the same content in two forms — one optimized for keyword indexing, one optimized for visual hierarchy. Google explicitly endorses the visually-hidden pattern in its accessibility guidelines.

Sitemap, robots, canonical, theme-color.

Four small files / tags that take ten minutes total and complete your SEO setup.

sitemap.xml

A list of every page on your site, with priority and changefreq. Help Google find everything.

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://jmartin.com/</loc>
    <changefreq>monthly</changefreq>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://jmartin.com/services/framing</loc>
    <changefreq>monthly</changefreq>
    <priority>0.8</priority>
  </url>
  ...etc
</urlset>
```

Save as `sitemap.xml` at the root of your site. Submit it to Google Search Console once.

robots.txt

```
User-agent: *
Allow: /
Sitemap: https://jmartin.com/sitemap.xml
```

Allows all crawlers, points to the sitemap. That's it.

Canonical link

```
<link rel="canonical" href="https://jmartin.com/">
```

On every page. Says "this is the official URL of this page." Prevents Google from getting confused by URL variants.

theme-color

```
<meta name="theme-color" content="#2F6779">
```

Colors the URL bar on iOS Safari. Tiny detail. Makes the site feel like an app.

The robots meta

```
<meta name="robots" content="index, follow, max-image-preview:large">
```

Allows indexing and following links, and tells Google it can use a large image preview in rich results.

PART EIGHT

Serverless APIs on Vercel.

Four chapters on the part of website-building most beginners avoid. How to capture a lead. How to wire a Claude-powered chat widget. How to gate a review request. Zero servers. Zero ops.

Lead capture — /api/lead.js.

Every contractor site needs a way to capture a name, phone, email, and message. Here's the exact serverless function I use, ready to copy.

A lead-capture form is the smallest, most useful piece of backend code most websites will ever need. Forty lines of JavaScript. Lives on Vercel. Sends an email or saves to a sheet. We're going to build it once and reuse it forever.

The function file

```
// api/lead.js
import { Resend } from 'resend';
const resend = new Resend(process.env.RESEND_API_KEY);

export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  const { name, phone, email, message } = req.body || {};

  if (!name || !phone) {
    return res.status(400).json({ error: 'Missing required fields' });
  }

  try {
    await resend.emails.send({
      from: 'leads@jmartin.com',
      to: 'jt@jmartin.com',
      subject: `New lead: ${name}`,
      text: `Name: ${name}\nPhone: ${phone}\nEmail: ${email}\n\n${message}`,
    });
    return res.status(200).json({ ok: true });
  } catch (err) {
    return res.status(502).json({ error: err.message });
  }
}
```


The form on the page

```
<form id="lead-form">
  <input name="name" placeholder="Your name"          required>
  <input name="phone" placeholder="Phone"              required>
  <input name="email" placeholder="Email"              type="email">
  <textarea name="message" placeholder="What can we help with?"></textarea>
  <button type="submit">Send</button>
</form>

<script>
document.getElementById('lead-form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const data = Object.fromEntries(new FormData(e.target));
  const res = await fetch('/api/lead', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });
  if (res.ok) e.target.reset();
});
</script>
```

What you need to set up

- 01 Sign up for Resend.**
Free tier covers 3,000 emails/month.
- 02 Verify your sending domain.**
One DNS record. Resend walks you through.
- 03 Add the API key to Vercel.**
Dashboard → Settings → Environment Variables → `RESEND_API_KEY` = your key.
- 04 Deploy.**
Vercel turns `api/lead.js` into `yoursite.com/api/lead` automatically.

Claude-powered concierge chat — /api/chat.js.

One of the single most useful upgrades you can make to a contractor's website. A chat bubble that answers FAQs, books appointments, and never sleeps. Powered by Claude Haiku for pennies a conversation.

I started adding Claude-powered chat to my client sites about six months ago. It is, without exaggeration, the upgrade clients react to the strongest. A homeowner lands on a roofer's site at 11pm, types "do you do flat roofs?" and gets a useful answer in real time. The same conversation in normal business hours would have been a missed-call-text-back, but with Claude the answer happens immediately.

The function file

```
// api/chat.js
import Anthropic from '@anthropic-ai/sdk';
const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

const KNOWLEDGE_BASE = `
You are the friendly assistant for JMartin Construction, a custom home
builder in Missoula, Montana. You answer questions about timber-frame
homes, our process, and our pricing. You always offer to schedule
a free consultation when the visitor seems serious.

Services: framing, siding, roofing, tile, interior trim, decks.
Service area: Missoula and Western Montana.
Pricing: Custom homes start at $400/sqft. Service work starts at $5,000.
Booking: https://jmartin.com/book

If you don't know something, say so and offer to connect them with JT.
Keep responses under 4 sentences unless asked for detail.
`;

export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  const { messages } = req.body || {};
  if (!Array.isArray(messages)) {
    return res.status(400).json({ error: 'messages array required' });
  }

  try {
    const result = await anthropic.messages.create({
      model: 'claude-haiku-4-5-20251001',
      max_tokens: 400,
      system: KNOWLEDGE_BASE,
      messages,
    });
    return res.status(200).json({ reply: result.content[0].text });
  } catch (err) {
    return res.status(502).json({ error: err.message });
  }
}
```

The widget on the page

A small fixed-position bubble in the corner. When clicked, opens a panel with messages and an input. Each message gets appended to a local array and POSTed to `/api/chat`. The reply gets rendered and the array grows.

The widget code is about 80 lines of vanilla JS. Claude can write it in one prompt: *"Build a chat widget in plain JS. Bottom-right fixed bubble. Opens a 360px panel. POSTs to /api/chat with { messages: [{role, content}] }. Renders the reply. Match the site's color palette."*

What it costs you

Claude Haiku is the cheapest of the three Claude tiers. At time of writing, a typical chat session of 6–8 messages costs less than a penny. A site that gets 200 chats a month costs you about \$1.50 in API fees. The increase in lead capture is, in my experience, 5–10x.

WHY THIS BEATS EVERY "AI CHAT" SAAS

Most AI chat SaaSes charge \$20–\$200/month per site. You're getting the same model — Claude — by writing 30 lines of code yourself. You own the prompt, you own the design, you own the data.

Gated review widget — `/api/review.js`.

Asks the customer how their experience was. 4 or 5 stars sends them to Google to leave a public review. 1–3 stars sends the feedback privately to the owner instead.

This is the single highest-ROI page on a contractor's website. It's the difference between a 4.2-star Google profile and a 4.9-star Google profile. Same customers. Same service quality. Different funnel.

How it works

1. You text the customer a link after a job: "Quick favor — how'd we do?"
2. The link opens a page with five stars.
3. If they tap 4 or 5 stars, they're redirected to your Google review URL.
4. If they tap 1, 2, or 3 stars, they get a private feedback form instead — which posts to `/api/review` and emails you.

Customers who had a great experience leave public reviews. Customers who didn't, you find out about privately and can save the relationship.

The full HTML page

I've open-sourced the page on every site I build. The pattern lives at `review-request/index.html` in my workspace. The key snippet:

```
const REDIRECT_URL = 'https://g.page/r/CeuzNJExNuIDEAE/review';

function onStar(stars) {
  if (stars ≥ 4) {
    window.location.href = REDIRECT_URL;
  } else {
    showPrivateForm(stars);
  }
}

async function submitFeedback(payload) {
  await fetch('/api/review', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
  });
}
```

The serverless function

```
// api/review.js
import { Resend } from 'resend';
const resend = new Resend(process.env.RESEND_API_KEY);

export default async function handler(req, res) {
  const { stars, comment, name, phone } = req.body || {};
  await resend.emails.send({
    from: 'reviews@jmartin.com',
    to: 'jt@jmartin.com',
    subject: `${stars}-star feedback from ${name || 'anonymous'}`,
    text: `Stars: ${stars}\nName: ${name}\nPhone: ${phone}\n\n${comment}`,
  });
  return res.status(200).json({ ok: true });
}
```

YELP SAYS DON'T DO THIS

Yelp's review guidelines forbid review gating. Google's, last I checked, don't — they discourage *incentivizing* reviews but the gated funnel is widely used. If you operate in a regulated industry or are nervous, ask all customers for a public review without filtering. Your call.

Environment variables — keeping keys out of git.

If you only learn one operational habit this whole book, make it this.
Never commit a key. Ever.

Every API key, password, and secret your site uses goes in environment variables. Not in code. Not committed to git. If you commit a key by accident, you assume someone hostile already has it — even private repos leak.

Local development — .env

Create a file called `.env` at the top of your project. Add your keys:

```
RESEND_API_KEY=re_your_key_here  
ANTHROPIC_API_KEY=sk-ant-your_key_here
```

Add `.env` to your `.gitignore` file so it's never committed. Most projects already have this.

Production — Vercel dashboard

Vercel doesn't read your local `.env`. You add production env vars in the dashboard:

1. Open your project in Vercel.
2. Settings → Environment Variables.
3. Add each key. Choose which environments (Production, Preview, Development) get it.
4. Redeploy. (Vercel doesn't re-apply env vars to existing deploys; you have to trigger a new build.)

Verifying — without exposing the keys

I never log a key. I log the *existence* of a key. In `server.js` or an API route:

```
if (!process.env.RESEND_API_KEY) {  
  console.warn('Missing RESEND_API_KEY');  
}
```

If you ever do leak a key to git, the only safe response is to rotate it. Old key gets revoked at the vendor, new key gets added to `.env` and Vercel. Don't try to clean the leaked key out of git history — it's already public to anyone who cloned the repo.

A REAL STORY

The single most expensive mistake I've made: I once committed an OpenAI key to a public repo. Within nine hours, someone had used it to rack up \$400 in API charges. Anthropic and OpenAI now scan public commits for keys and revoke them automatically, but don't count on it.



PART NINE

Shipping it.

vercel.json, clean URLs, security headers, custom domains, preview deploys. The unglamorous five percent of the work that separates a hobby project from a production website.

vercel.json — the only config file you need.

A single JSON file at the root of your project tells Vercel how to build, route, and deploy. Most sites need ten lines. Here's the full vocabulary.

The minimum vercel.json — for a plain HTML site

```
{
  "cleanUrls": true,
  "trailingSlash": false
}
```

That's it. Clean URLs means `/about` serves `about.html` — no `.html` in the URL. Trailing slash `false` means `/about/` redirects to `/about`. Both are SEO best practice.

For a React + Vite site

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [
    { "source": "/((?!api).*)", "destination": "/index.html" }
  ]
}
```

The rewrite catches every URL except `/api/*` and serves `index.html` — so client-side routing works on direct links.

For a multi-project monorepo

Like my workspace, which has 10+ sites. The root `vercel.json` tells Vercel which subdirectory is the primary site:

```
{
  "buildCommand": "npm install && cd jtfasulo-website && npm install && npm run build",
  "outputDirectory": "jtfasulo-website/dist",
  "rewrites": [
    { "source": "/((?!api).*)", "destination": "/index.html" }
  ]
}
```

Adding redirects

```
{
  "redirects": [
    { "source": "/services", "destination": "/pricing", "permanent": true },
    { "source": "/blog/old-post", "destination": "/blog", "permanent": true }
  ]
}
```

Use `"permanent": true` for 308 redirects (search engines will move the SEO juice). Use `"permanent": false` for 307 temporary redirects.

Adding security headers

```
{
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
        { "key": "Permissions-Policy", "value": "geolocation=(), microphone=(), camera=()"}
      ]
    }
  ]
}
```

These three headers cover 80% of the basic security wins. Add to every production site.

Clean URLs, redirects, security headers.

Deep dive on the three things that signal "this site is production-grade" — to Google, to browsers, and to you.

Clean URLs

The difference between `/services/framing.html` and `/services/framing` is small to look at, big in practice. Google indexes both as separate URLs unless you tell it not to. The cleaner URL is the one you want in the SERP. The cleaner URL is the one to put on business cards.

With Vercel's `cleanUrls: true`, any `.html` file at `/services/framing.html` is automatically served at `/services/framing`. The `.html` version 308-redirects to the clean version.

Redirects — the slow-loss problem

Every URL change you make should have a redirect. Rename a page from `/contact-us` to `/contact`? Add a 308 redirect. Move a blog post? Redirect. Otherwise:

- Old links you can't update (Yelp profiles, business cards, Google My Business) point to 404 pages.
- You lose the SEO authority the old URL had accumulated.
- Customers think your site is broken.

Security headers

Three I add to every site:

- **X-Content-Type-Options: nosniff** — stops browsers from "guessing" file types, a common attack vector.
- **Referrer-Policy: strict-origin-when-cross-origin** — privacy. Don't leak referrer URLs to external sites.
- **Permissions-Policy: geolocation=(), microphone=(), camera=()** — explicitly say "this site does not need any of these."

I learned this from JMartin Construction's deploy. After turning these on, my Lighthouse "Best Practices" score went from 87 to 100, and so did the perceived professionalism of the site.

Connecting a custom domain.

Buy a domain. Point it at Vercel. Wait fifteen minutes. Done.

A custom domain is the difference between `jmartin-construction.vercel.app` and `jmartin.com`. It costs \$12–\$30 a year and takes fifteen minutes to set up.

Step one — buy the domain

I use **Porkbun** or **Cloudflare Registrar**. Both are at-cost (no markup), have clean UIs, and let you transfer in and out without games. Avoid GoDaddy (overpriced, aggressive upsells), Squarespace Domains (locked-in DNS), and Namecheap's free WHOIS-privacy bait-and-switch.

Step two — add it in Vercel

1. Open your project in Vercel.
2. Settings → Domains.
3. Type your domain. Hit Add.
4. Vercel shows you two DNS records to add: an A record and a CNAME.

Step three — point DNS at Vercel

At your registrar (Porkbun, Cloudflare, etc.), open DNS settings for the domain. Add the records Vercel showed you:

Type	Name	Value
A	@	76.76.21.21
CNAME	www	cname.vercel-dns.com

Save. Wait. Within fifteen minutes, your domain points at Vercel and Vercel automatically provisions a free SSL certificate (the lock icon in the browser bar).

Step four — verify

Open your new domain in an incognito window. You should see your site. If you see "Site not found" or a SSL warning, wait another fifteen minutes — DNS can take up to an hour to propagate.

APEX VS WWW

I always set up both. `jmartin.com` (apex) is the canonical. `www.jmartin.com` 308-redirects to the apex. Configure both in Vercel's domain panel. Vercel handles the redirect for you.

Preview deploys — the safety net you're not using.

Every branch, every PR, every commit gets its own URL. Use it to share work, test risky changes, and get client feedback without touching production.

Vercel's preview deploys are one of the most underused features I see. Most people only think about production. But every branch is a preview, automatically, with its own URL. Use them.

The pattern I use

- 01 **Create a branch.**
`git checkout -b new-pricing-page`
- 02 **Build the change.**
Anything risky — a redesign, a new section, a new tool integration.
- 03 **Push the branch.**
`git push -u origin new-pricing-page`
- 04 **Vercel builds a preview.**
Within seconds, you get a URL like `jmartin-git-new-pricing-page-jt.vercel.app`.
- 05 **Send the URL to the client.**
They see the change on a real URL, on their phone, before you've touched the live site.
- 06 **Approved? Merge the branch.**
`git checkout master && git merge new-pricing-page && git push`. Vercel rolls the change to production.
- 07 **Rejected? Just don't merge.**
The preview URL exists forever (or until you delete the branch); production never sees the change.

Why this matters

You stop having to ask "should I deploy this?" because deploying isn't dangerous — production only updates when you merge. You stop having to email screenshots to clients because they can see the change live, on their own device, at a real URL.

I've shipped redesigns of clients' homepages without them ever seeing the old version interrupted. Preview deploy → review → merge → live. Zero downtime. Zero anxiety.



PART TEN

A full walkthrough.

Three days. Cold start. A real plumber's website, built from one prompt to a live URL. Every command. Every file. Every commit. The whole loop, end to end.

Day 1 — from first prompt to a finished hero.

Friday morning. New project. By Friday afternoon, a hero section that makes the client cry happy tears.

Imagine a plumber. We'll call him Mike. Mike runs *Reliable Plumbing* in Bend, Oregon. He has a Wix site that hasn't been updated in five years. He asks me to build him something new. Friday, 9am, I open the terminal.

9:00 am — the folder

```
cd ~/Sites
mkdir reliable-plumbing
cd reliable-plumbing
claude
```

9:02 am — the first prompt

VERBATIM — THE PROMPT

"I'm building a website for Reliable Plumbing, a plumber in Bend, Oregon. Family-owned. 20 years in business. Real people, not corporate. Voice: warm, direct, Bend-y. Reference sites: the no-nonsense feel of Bartsian.com, the typography of the Patagonia worn-wear site.

Stack: plain HTML, vanilla JS, no build step. Pages: home, services (drains, water-heaters, repipes, emergencies), about, contact. Color palette: deep navy, warm cream, one bright copper accent.

Give me a plan first. Don't write code yet."

9:04 am — Claude's plan

Claude responds with a numbered list:

1. Create `index.html` with header, hero, services preview, testimonials, contact CTA, footer.
2. Create four service pages with shared design.
3. Create `styles.css` with the three-color palette as CSS variables.
4. Create `main.js` for the mobile nav toggle.

5. Create `api/lead.js` for the contact form.
6. Create `vercel.json` with cleanUrls and basic security headers.

I approve the plan. Two corrections: "use Cormorant Garamond for the display font" and "skip `main.js` for now — we'll add JS only when we need it."

9:08 am — Claude writes

While Claude works, I open Higgsfield in a tab and queue up three hero clips: a slow tracking shot of a kitchen sink with copper pipes, a wide shot of a wrench on a workbench, and a pan across a finished bathroom remodel. Each takes ~10 minutes to generate.

9:30 am — the hero is in

I look at the hero. Tight headline, navy background, copper accent. Decent. But the CTA looks too startup-y — a rounded purple button that doesn't fit. I say: *"Replace the CTA with a hard-edged copper button. 4px radius. Bold uppercase 0.18em letter-spacing label that says 'Call now — (541) 555-0100'."*

Claude updates. Now the hero looks like the brand. We're 30 minutes in.

10:00 am — wire up Git and Vercel

```
git init
git add .
git commit -m "Reliable Plumbing — initial homepage with hero + skeleton sections"
gh repo create reliable-plumbing --public --source=. --push
vercel link
vercel --prod
```

Site is live at `reliable-plumbing.vercel.app`. I text Mike the URL. We're still on Friday, mid-morning, with seven hours of build time left.

The rest of Day 1

By dinnertime: hero polished, services skeleton, testimonials skeleton, contact form wired, vercel.json done, mobile pass done, one Higgsfield clip embedded in the hero, OG image generated. Committed and deployed eight times. Day one of three.

Day 2 — services, testimonials, contact, footer.

The wide sweep. By Saturday night, every section exists at the polish level of the hero.

Day two is the section pass. I take every part of the page that's a placeholder and bring it to the standard the hero set.

10:00 am — services

Four service pages. Same shell. Different copy and imagery. I write the copy myself (or pull it from Mike's old Wix site) and paste it into a `copy.md` file in the project. Then:

PROMPT

"Build four service pages: drains, water-heaters, repipes, emergencies. Use `copy.md` for the body content. Each page: hero with the service name, three paragraphs of copy, a 'common problems we fix' list, a 'why us' subsection, a CTA. Match the homepage's design — same nav, same footer, same type hierarchy."

Claude builds all four in one shot. Total time: 12 minutes.

11:00 am — testimonials

I pull five Google reviews from Mike's existing profile, paste them into `copy.md`, and ask Claude to build a testimonials section. *"Three columns desktop. One column mobile. Plain quote, name, date. No avatar. Hairline border between."*

1:00 pm — the contact section

The form from Chapter 29, copy-pasted into `api/lead.js`. Wire it up to the form on `/contact`. Test it once. Boom — the form sends a real email to Mike's inbox. He's excited.

3:00 pm — footer + nav polish

Three-column footer: services, contact info, hours. Sticky nav with the brand wordmark, three links, and the phone number as a tap-to-call. Mobile nav opens a full-screen overlay with the same links.

5:00 pm — copy pass

I read every word top to bottom. Anything that sounds like ChatGPT, I rewrite. The most common things I edit out: "comprehensive solutions," "best-in-class," "leveraging," "synergize," anything ending in *-driven*. By the end of the pass the site sounds like Mike — direct, warm, slightly sarcastic about the bigger plumbing companies in town.

7:00 pm — commit & deploy

Three commits over the day. One after services. One after testimonials. One after the copy pass. Each one pushes to GitHub, each one triggers a Vercel deploy, each one updates the live URL.

End of Day 2

The site is structurally done. Every section exists. Every page has the same level of polish as the hero. What remains: mobile pass, perf pass, SEO pass, and the final design pass to make everything feel like one thing.

Day 3 — mobile pass, perf pass, deploy.

Sunday. The last 5% that takes 50% of the perceived quality. By 6pm, Mike has a production website at reliablebend.com.

Day three is the day a Wix site never gets. The mobile pass. The perf pass. The SEO pass. The polish. The custom domain. By the end of Sunday Mike has a website that outperforms three competitors that paid \$8,000 to an agency.

10:00 am — mobile pass

Chrome DevTools, "Toggle device toolbar." iPhone SE, then iPhone 14, then a Pixel 7, then an iPad. Walk every section.

THINGS I FIND

The hero h1 is too big at 380px wide — wraps awkwardly. Fix: tighter clamp. The services grid is 2 columns at 480px — should be 1. Fix: media query. The footer has the phone number as small text — should be a big tap target. Fix: bigger phone-number block on mobile.

Each finding is one short prompt to Claude. Each fix takes 30 seconds. Whole mobile pass is done in 45 minutes.

11:30 am — perf pass

Open `pagespeed.web.dev`, paste the URL. Run on mobile and desktop. Look for any red or yellow.

Typical findings: hero video too big (compress with the `ffmpeg` command from Chapter 23), one image not in WebP (convert), one font weight not used but loaded (remove). Twenty minutes of fixes, scores jump from 78 to 96.

12:30 pm — SEO pass

Walk every page. Title tag pattern from Chapter 24. Meta description, OG, Twitter card from Chapter 25. JSON-LD LocalBusiness schema from Chapter 26. Sitemap.xml, robots.txt, canonical, theme-color from Chapter 28. The whole pass takes 35 minutes.

2:00 pm — domain

Mike buys reliablebend.com at Porkbun. He gives me login. I add the DNS records Vercel asked for. Fifteen minutes later reliablebend.com resolves to the live site, with a green padlock.

3:00 pm — handoff

I screen-record a five-minute Loom for Mike: how to update the testimonials in `copy.md`, how to add a service, who to call if something breaks. I send him a final invoice. He pays the same day.

6:00 pm — done

Three days. One terminal. \$44 in tools. A website Mike will look at for the next ten years.

Ten mistakes I made so you don't have to.

Every one of these cost me a day or more. Don't repeat them.

- 01 Committing a key to a public repo.**
\$400 in fraudulent API charges. Use `.env`. Add it to `.gitignore`. Rotate keys monthly.
- 02 Skipping the plan step.**
The first few months I'd say "build me a hero" without a plan. Got 200 lines of code I didn't want. Always ask for a plan first.
- 03 Importing 21st.dev components verbatim.**
Two failed deploys and a slow site later, I learned to rewrite by hand. Chapter 18.
- 04 Using `requestAnimationFrame` for marquees.**
Lag on Safari. Lag on any older laptop. CSS `@keyframes` do the same job for free.
- 05 Forgetting the `poster` attribute on hero video.**
4-second blank screen on every page load. Always add a poster.
- 06 Two `h1`s on one page.**
SEO confusion. One `h1` per page. Use `h2/h3` for the rest. (Hidden-`h1` trick is the only exception — see Chapter 27.)
- 07 Skipping the mobile pass on desktop-only design days.**
Discovered three days later that the entire site overflowed at 380px. Now I never skip the mobile pass.
- 08 Treating Claude like a search engine.**
"How do I build a hero?" — wasted week. "Build me a left-aligned hero with..." — done in twenty minutes.
- 09 Not committing for a whole day.**
Browser crashed. Lost the work. Now I commit after every meaningful change.
- 10 Pre-optimizing.**
Tried to set up Cloudflare R2 before I had a single visitor. Six hours of yak shaving. Start on Vercel free tier. Upgrade when you must.

What I learned building ten of these.

A reflective close, with the patterns I'd never have caught from any single project.

Ten sites in, the patterns stop being patterns and start being instincts. Here are the things I didn't know I knew until I tried to write this book.

Voice is harder than design

Sites look distinctive when their copy sounds distinctive. The visual design is half the work. The other half is the headlines, eyebrows, button labels, and footer microcopy. I now spend equal time on each. Three sentences in the right voice beats three hours of fancy gradients.

The mobile experience is the product

Roughly 70% of my contractor sites' traffic is mobile. The desktop version is the audition. The mobile version is the show. I now build mobile-first on every project.

Small, frequent commits are how you stay sane

The single habit that's saved me the most pain. I never get more than 20 minutes from my last commit. If something breaks, I revert one thing — not a day.

The boring stuff matters most

SEO. Performance. Security headers. The boring chapter (Chapter 28) is the one that produces inbound leads months later. The visual chapter (Chapter 14) is the one that gets reactions today. You need both.

Tools are interchangeable; thinking isn't

I've used Claude Code, Cursor, GitHub Copilot, ChatGPT, and a half-dozen others. The tool that's "best this month" rotates. The thinking — passes, symptoms, constraints, plans before code, commit early, mobile-first — is invariant. Master the thinking and the tool stops mattering.

"The output of your build session is shaped less by the tool than by the instructions you give it. Get the instructions right, and almost any AI will write a great website. Get the instructions wrong, and the best AI in the world will produce slop."

You will not love every site you build

Some clients want what they want. The site you ship will sometimes look less like your portfolio piece and more like their hand-drawn napkin. That's fine. Ship anyway. Cash the check. Build the next one.

The job is mostly conversation

I'm a builder. Half my time is in the terminal. The other half is in Loom, on email, on the phone, talking to the people the site exists for. The terminal can only output what you input. So master the inputting — by listening to the customer, then translating the listening into instructions.

Where to go from here — resources and links.

A compact, opinionated list of every tool, doc, and creator I'd send a beginner toward.

Docs to bookmark

- **Claude Code docs** — docs.anthropic.com/claude-code
- **Vercel docs** — vercel.com/docs — the most readable platform documentation I've ever encountered.
- **MDN** — developer.mozilla.org — the authoritative reference for everything HTML / CSS / JS.
- **Schema.org** — schema.org/docs/full.html — for every JSON-LD type.

People to follow

- The Anthropic blog for Claude updates.
- The Vercel changelog for product news.
- Anything Brad Frost has written about design systems.
- Adam Wathan's old blog posts on Tailwind utility CSS.
- Refactoring UI (the book) for visual taste calibration.

Tools I didn't fit in but use

TOOL	WHAT FOR	COST
Resend	Transactional email from /api/* functions	Free (3k/mo)
Stripe	Payment links + checkout	2.9% + 30¢/tx
Cloudflare R2	Cheap object storage for video / large assets	~\$0.015/GB
Loom	Client walkthroughs and handoffs	Free tier
Notion	Project briefs, content drafts	Free
Figma	OG-image design, the occasional mockup	Free tier

Want to work with me?

If you've made it this far and you want help building a site — or you want a done-for-you Sybago site for your business — I'm at jtfasulo.com. Book a call. Or buy the next version of this guide when it comes out.

One last thing

You bought this guide because you wanted to build a website with Claude Code. So go build one. Don't read this twice. Don't research five more tools. Don't watch eight YouTube videos. Open a terminal, make a folder, type `claude`, and ship something embarrassing by Friday. The first one's the hardest. The tenth one's a habit.

© 2026 JT Fasulo · Sybago · jtfasulo.com · Single-reader license. Not for redistribution.